OO Modeling Practices

Main Idea

Modeling helps us understand a problem before writing code. We identify objects, their information, an their actions. A model shows important details and leaves out innecessary details.

1:- Objects and responsibilities

An object represents something in our system.
Each class should have a clear responsibility.
Example: a printing business has costumers, orders, and payments.

Costumer: stores costumer information
WorkOrder: manages the requested job.
Payment: represents money paid for an order.

2:- Cohesion and coupling

Cohesion: the parts of a class work toward the same purpose. high cohesion makes a class easier to understand
Coupling: classes depend on eauch other.
Avoid unnessegary dependencies. A change in on class can affect another.

3:- Clear names

Use names that explain their purpose.
Example:
Class: Costumer - names a thing
Attribute: phoneNumber - names information
Method: CalculeTotal() - names an action

4:- Organize and test

Separate the user interface, business rules, and data storage.
This makes changes easier.

5:- Small class example

```
class WorkOrder () {
        private double total = 100;
        private double paid = 40;
        public double calculateBalance () {
                return total - paid;
        }
}
```

Notas

Escaneado con CamScanner

Introduction to UML

What is UML?

U.M.L = Unified Modeling language.

it is a visual language for describing systems.
it uses boxes, lines, arrows, and other symbols.
UML is not a programming language.

Why do we use it?

→ Understand a problem.
→ Plan a solution.
→ Explains ideas to other people
→ Document a system.

Main diagrans

Class diagram:
Shows classes and their relationships
Example: Costumer, WorkOrder and Pegment

object diagram:
shows specific object at one moment
Example: Costumer 1, with the name Ana.

Different Perspetives

→ **Analysis:** Underland the real problem.
→ **Specification:** describe what the software should do
→ **Implenentation:** describe how the software is built

Good Diagram habits

Use clear titles, Keep related elements together. Avoid crosing lines and unnecessary details

Escaneado con CamScanner

Basic Uses Cases

Main Idea

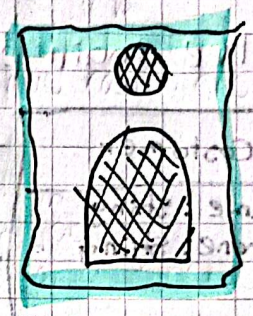
A use cases describes a goal that an actor wants to achieve. It explains what the actor needs from the systems.

Actor → Goal → Use case

What is a Actor?

An actor is outside the system and interacts with it it can be a person, another system, or a device.

And Actor represents a role, not one specific, perso, several people can have the same role.



What is a Use Case?

A use case represents a useful goal. Give it a short name with an action

Examples:
- → Registrer Costumers.
- → Create Work Order.
- → Record Payment.

Goal or interaction step?

Goal: Record Payment

Steps: Select an order, enter an amount, and confirm.

The use case explains what can why, the interation steps explaind how the user works with the system. A scenario is one specific was a use case happens.

Diagram Symbols

- → Stick: actor.
- → Oval: use case.
- → Rectangle: System boundary
- → Solid line: connects an actor to use case.

Relationships

- → Include: required behavior another use case.
- → Extend: Extra behavior under certain conditions
- → Generalization: a more specific version of an actor or use case.

Escaneado con CamScanner

Class Diagrams

Class and Object

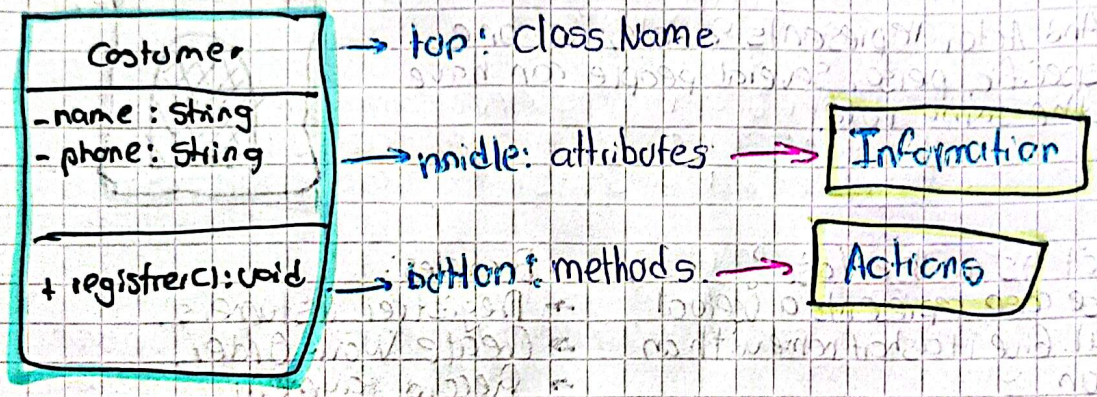
A class describes objects with common information and behavior.

An object is one specific inlance of a class

Example: Costumer

Example: Costumer 1

The three main parts



Visibility

→ + Public: can be accessed from outside the class.

→ - Private: belongs to the class's internal details.

→ # protected: allows in acces in the same package

Names and Relationships

Class names use UpperCamelCase: WorkOrden

Attributes and methods use lowerCamelCase: calculateTotal().

Relationships connect classes. A dependency means that one class relies on another.

Escaneado con CamScanner

UML Quick Reference

Main idea:

Choose the right diagram for the idea you want to explain. Keep diagrams simple, readable, and focused.

Class relationships

Association:
objects have a conection
Example: a costumer places an oder

Dependency:
One class relies on another
Example: Draw a dashed arrow toward the class being used.

Generalization:
an "is a" relationship
Example: a graduate studen is a student

Aggregation:
A whole contains parts.

Composition:
A stronger whole-and-part relationship
The parts share the lifetime of the whole.

Multiplicity

- 1 = exactly one.
- 0..1 = zero or one.
- 0..* = zero or many.
- 1..* = one or many.

Activity Diagrams

they shows actions, decisions and process flow

Interaction diagrams

They show objects exchanging messages. A sequence emphasizes time order. A communication digram emphasizes connections between objects.

State diagrams

they shaw cases and transitions
An event can cause an obsect to chage state

Good habits

Use clear names, avoid crossing lines, and include only useful details. During analysis, focus on the problem before technical solutions.

Notes

Escaneado con CamScanner